Summary

Hooks are a new feature addition in React version 16.8

They allow you to use React features without having to write a class

Avoid the whole confusion with 'this' keyword

Allow you to reuse stateful logic

Organize the logic inside a component into reusable isolated units

Rules of Hooks

"Only Call Hooks at the Top Level“

Don’t call Hooks inside loops, conditions, or nested functions

"Only Call Hooks from React Functions“

Call them from within React functional components and not just any regular JavaScript function

Summary - useState

The useState hook lets you add state to functional components

In classes, the state is always an object.

With the useState hook, the state doesn't have to be an object.

The useState hook returns an array with 2 elements.

The first element is the current value of the state, and the second element is a state setter function.

New state value depends on the previous state value? You can pass a function to the setter function.

When dealing with objects or arrays, always make sure to spread your state variable and then call the setter function

Consuming Multiple Contexts

```
function Content() {
  return (
    <ThemeContext.Consumer>
      {theme => (
        <UserContext.Consumer>
          {user => (
            <ProfilePage user={user} theme={theme} />
          )}
        </UserContext.Consumer>
      )}
    </ThemeContext.Consumer>
  );
}
```

useReducer

useReducer is a hook that is used for state management

It is an alternative to useState

What's the difference?

useState is built using useReducer

Hooks so far

useState – state

useEffect – side effects

useContext – context API

useReducer - reducers

JavaScript Demo: Array.reduce()

```
1  const array1 = [1, 2, 3, 4];
2  const reducer = (accumulator, currentValue) => accumulator + currentValue;
3
4  // 1 + 2 + 3 + 4
5  console.log(array1.reduce(reducer));
6  // expected output: 10
7
8  // 5 + 1 + 2 + 3 + 4
9  console.log(array1.reduce(reducer, 5));
10 // expected output: 15
11
```

reduce vs useReducer

reduce in JavaScript	useReducer in React
array.**reduce**(***reducer***, initialValue)	**useReducer**(***reducer***, initialState)
singleValue = ***reducer***(accumulator, itemValue)	newState = ***reducer***(currentState, action)
reduce method returns a single value	**useReducer** returns a pair of values. [newState, dispatch]

useState vs useReducer

Scenario	useState	useReducer
Type of state	Number, String, Boolean	Object or Array
Number of state transitions	One or two	Too many
Related state transitions?	No	Yes
Business logic	No business logic	Complex business logic
Local vs global	Local	Global